

SƯỜN ÔN TẬP TRỌNG TÂM: TRÌNH BIÊN DỊCH

Tập trung: Phân tích từ vựng & Phân tích cú pháp LL(1)

Cập nhật dựa trên bài giảng 2012

Mục lục

1	Tổng quan các giai đoạn Biên dịch	2
2	Chương 1: Phân tích từ vựng (Lexical Analysis)	2
2.1	Các khái niệm cơ bản	2
2.2	Biểu thức chính quy (Regular Expressions - RE)	2
2.2.1	Các khối nguyên tố (Elementary RE)	2
2.2.2	Các phép toán phức hợp (Complex RE)	3
2.2.3	Các ký hiệu mở rộng (Dùng trong công cụ Flex)	3
2.3	Xử lý các khó khăn và sự mơ hồ	3
2.3.1	Quy tắc giải quyết đụng độ	3
2.4	Xử lý lỗi từ vựng	3
3	Các dạng bài tập Chương 1	4
4	Chương 2: Phân tích cú pháp (Syntax Analysis)	4
4.1	Văn phạm phi ngữ cảnh (CFG)	4
4.2	Dẫn xuất và Cây cú pháp	4
4.3	Các dạng bài tập trọng tâm Chương 2	4
5	Kỹ thuật Phân tích LL(1)	10
5.1	Điều kiện để một văn phạm là LL(1)	10
5.2	Các quy tắc cốt lõi tính toán tập hợp LL(1)	10
5.2.1	Quy tắc tìm tập FIRST	10
5.2.2	Quy tắc tìm tập FOLLOW	10
5.2.3	Quy tắc lập Bảng phân tích cú pháp LL(1)	11
5.3	Thuật toán mô phỏng Stack phân tích LL(1)	11
5.4	Hệ thống Bài tập Phân tích và So khớp Chuỗi (Theo mẫu bài giảng)	11

1 Tổng quan các giai đoạn Biên dịch

Cần ghi nhớ sơ đồ 7 giai đoạn để xác định vị trí của từng quá trình:

1. **Phân tích từ vựng (Lexical Analysis):** Đọc luồng ký tự, nhóm thành các Token.
2. **Phân tích cú pháp (Syntax Analysis):** Kiểm tra cấu trúc dựa trên văn phạm (CFG), tạo cây cú pháp.
3. **Phân tích ngữ nghĩa (Semantic Analysis):** Kiểm tra kiểu dữ liệu, khai báo biến.
4. **Sinh mã trung gian (IR Generation).**
5. **Tối ưu mã trung gian.**
6. **Sinh mã máy (Code Generation).**
7. **Tối ưu mã máy.**

2 Chương 1: Phân tích từ vựng (Lexical Analysis)

2.1 Các khái niệm cơ bản

- **Token (Dấu hiệu/Loại từ):** Là một cặp bao gồm tên token và giá trị thuộc tính (tùy chọn). Nó đại diện cho một đơn vị logic.
 - Ví dụ: `T_While`, `T_Identifier`, `T_Plus`.
- **Lexeme (Từ vị):** Là chuỗi ký tự cụ thể trong chương trình nguồn khớp với mẫu của một token.
 - Ví dụ: Lexeme `"while"` khớp với token `T_While`; lexeme `"137"` khớp với token `T_IntConst`.
- **Attribute (Thuộc tính):** Thông tin bổ sung dùng cho các giai đoạn sau (như Phân tích ngữ nghĩa).
 - Ví dụ: Thuộc tính của `T_IntConst` là giá trị số của nó (137), thuộc tính của `T_Identifier` là tên biến hoặc vị trí trong bảng ký hiệu.

2.2 Biểu thức chính quy (Regular Expressions - RE)

RE là công cụ toán học dùng để mô tả tập hợp các lexemes của một token.

2.2.1 Các khối nguyên tố (Elementary RE)

- ϵ : Biểu thức chính quy cho chuỗi rỗng.
- a : Biểu thức chính quy cho ký tự 'a'.

2.2.2 Các phép toán phức hợp (Complex RE)

Giả sử R và S là các RE:

- **Ghép nối (Concatenation):** RS (Chuỗi thuộc R theo sau bởi chuỗi thuộc S).
- **Hợp (Union):** $R|S$ hoặc $R \cup S$ (Chuỗi thuộc R hoặc thuộc S).
- **Bao đóng Kleene (Kleene Star):** R^* (Không hoặc nhiều lần lặp lại R).
- **Thứ tự ưu tiên:** $(R) > R^* > RS > R|S$.

2.2.3 Các ký hiệu mở rộng (Dùng trong công cụ Flex)

- $.$: Bất kỳ ký tự nào (ngoại trừ xuống dòng).
- $[abc]$: Ký tự 'a' hoặc 'b' hoặc 'c'.
- $[0-9]$: Ký tự số từ 0 đến 9.
- $[^abc]$: Bất kỳ ký tự nào TRỪ 'a', 'b', 'c'.
- r^+ : Một hoặc nhiều lần r (tương đương rr^*).
- $r?$: Không hoặc một lần r (tùy chọn).
- $r\{n,m\}$: Ít nhất n lần và tối đa m lần r .

2.3 Xử lý các khó khăn và sự mơ hồ

- **Vấn đề khoảng trắng:** Trong một số ngôn ngữ cũ như FORTRAN, khoảng trắng không quan trọng (DO 5 I giống DO5I), gây khó khăn cho việc tách token.
- **Sự mơ hồ (Ambiguity):** Khi một chuỗi ký tự khớp với nhiều định nghĩa token (Ví dụ: `double` có thể là từ khóa hoặc là tên biến `id`).

2.3.1 Quy tắc giải quyết đụng độ

1. **Maximal Munch (Tiêu thụ tối đa):** Luôn chọn chuỗi dài nhất có thể khớp được.
 - Ví dụ: Với đầu vào `"fort"`, trình phân tích sẽ chọn nó là một định danh (`id`) thay vì từ khóa `for` và ký tự `t`.
2. **Độ ưu tiên luật (Priority):** Nếu hai luật cùng khớp với một độ dài chuỗi như nhau, luật nào được định nghĩa trước sẽ thắng.
 - Ví dụ: Định nghĩa từ khóa `if` trước định danh `id` để đảm bảo chuỗi `"if"` được nhận diện là từ khóa.

2.4 Xử lý lỗi từ vựng

Nếu không có luật nào khớp với chuỗi nhập, ta sử dụng một luật **"catch-all"**:

- Dùng ký hiệu $.$ (dấu chấm) ở cuối danh sách các luật để khớp với bất kỳ ký tự đơn lẻ nào chưa được xử lý và thông báo lỗi *"Unrecognized character"*.

3 Các dạng bài tập Chương 1

Dạng 1: Viết Biểu thức chính quy:

- Ví dụ: Viết RE mô tả số thực có dấu: $(+|-)?[0-9]+(\.[0-9]+)?(E(+|-)?[0-9]+)?$

Dạng 2: Phân tích chuỗi bằng Maximal Munch:

- Cho tập luật (1: for, 2: id, 3: number). Phân tích chuỗi "fort10".
- *Giải:* "fort" khớp dài nhất với id, sau đó "10" khớp với number. Kết quả: `<id, "fort"> <number, 10>`.

4 Chương 2: Phân tích cú pháp (Syntax Analysis)

4.1 Văn phạm phi ngữ cảnh (CFG)

Được định nghĩa bởi bộ tứ (V, Σ, P, S) :

- V : Tập các ký hiệu chưa kết thúc (Non-terminals).
- Σ : Tập các ký hiệu kết thúc (Terminals/Tokens).
- P : Tập các luật sinh $(A \rightarrow \alpha)$.
- S : Ký hiệu bắt đầu.

4.2 Dẫn xuất và Cây cú pháp

- **Dẫn xuất trái nhất (Leftmost Derivation):** Luôn khai triển ký hiệu chưa kết thúc nằm ngoài cùng bên trái.
- **Văn phạm mơ hồ:** Khi một chuỗi có > 1 cây phân tích cú pháp.
- **Phân tầng văn phạm:** Cách để khử mơ hồ bằng cách chia mức ưu tiên toán tử (VD: E (cộng/trừ) $\rightarrow T$ (nhân/chia) $\rightarrow F$ (nguyên tố)).
- **AST (Abstract Syntax Tree):** Cây rút gọn, chỉ giữ lại các node mang ý nghĩa thực thi (toán tử, toán hạng).

4.3 Các dạng bài tập trọng tâm Chương 2

Dạng 1: Tìm dẫn xuất trái nhất (Leftmost Derivation)

- **Phương pháp giải:**
 - Ở mỗi bước thế, luôn quét từ trái sang phải và chỉ chọn biến (ký hiệu chưa kết thúc) đầu tiên bắt gặp nằm ở vị trí ngoài cùng bên trái để áp dụng luật sinh.
 - Ký hiệu biến được chọn thường được gạch chân để tránh nhầm lẫn trong quá trình làm bài.

- **Bài tập mẫu 2.1:** Cho văn phạm phẳng G (chưa phân tầng) có tập luật sinh P :

$$E \rightarrow E + E \mid E \times E \mid (E) \mid \text{id}$$

Hãy tìm dẫn xuất trái nhất ($\xrightarrow{*}_{lm}$) để sinh ra chuỗi: $w = \text{id} + \text{id} \times \text{id}$ theo đúng quy ước toán học thông thường (tức là tính nhân trước, cộng sau).

- **Lời giải chi tiết:** Để đảm bảo quy ước toán học (phép nhân được gom cụm tính trước, phép cộng tính sau), trên cây cú pháp phép toán cộng phải nằm ở tầng cao hơn (gần gốc hơn). Do đó, bước đầu tiên ta phải chọn luật sinh chứa phép cộng:

$$\begin{array}{ll} E \xrightarrow{lm} \underline{E} + E & (\text{Chọn luật } E \rightarrow E + E) \\ \xrightarrow{lm} \text{id} + \underline{E} & (\text{Thế biến trái nhất } \underline{E} \rightarrow \text{id}) \\ \xrightarrow{lm} \text{id} + \underline{E} \times E & (\text{Thế biến trái nhất } \underline{E} \rightarrow E \times E) \\ \xrightarrow{lm} \text{id} + \text{id} \times \underline{E} & (\text{Thế biến trái nhất } \underline{E} \rightarrow \text{id}) \\ \xrightarrow{lm} \text{id} + \text{id} \times \text{id} & (\text{Thế biến cuối cùng } \underline{E} \rightarrow \text{id}) \end{array}$$

Dạng 2: Chứng minh văn phạm mơ hồ (Ambiguous Grammar)

- **Phương pháp giải:**

- Để chứng minh một văn phạm là mơ hồ, ta cần chọn một chuỗi cụ thể thuộc ngôn ngữ và chỉ ra chuỗi đó có **2 dẫn xuất trái nhất (lm) khác nhau**, dẫn đến việc tồn tại nhiều hơn một cây phân tích cú pháp.
- **Ý nghĩa ngữ nghĩa:** Mỗi cây cú pháp tương ứng với một cách hiểu thứ tự tính toán khác nhau. Văn phạm mơ hồ khiến trình biên dịch không thể xác định được ý nghĩa chính xác của biểu thức.

- **Bài tập mẫu 2.2:** Hãy chứng minh văn phạm $E \rightarrow E + E \mid E \times E \mid (E) \mid \text{id}$ là một văn phạm mơ hồ.
- **Lời giải chi tiết:** Ta chọn chuỗi chứng minh là $w = \text{id} + \text{id} \times \text{id}$. Ta chỉ ra 2 cách dẫn xuất trái nhất hoàn toàn khác nhau cùng sinh ra chuỗi này:

- **Dẫn xuất trái nhất 1 (Diễn giải ĐÚNG toán học - Nhân trước, Cộng sau):** Luôn chọn luật cộng trước để đẩy phép cộng lên gốc (tính sau cùng):

$$E \xrightarrow{lm} \underline{E} + E \xrightarrow{lm} \text{id} + \underline{E} \xrightarrow{lm} \text{id} + \underline{E} \times E \xrightarrow{lm} \text{id} + \text{id} \times \underline{E} \xrightarrow{lm} \text{id} + \text{id} \times \text{id}$$

- **Dẫn xuất trái nhất 2 (Diễn giải SAI toán học - Ép toán tử cộng tính trước):** Xuất phát bằng việc chọn luật sinh chứa phép nhân trước, ép phép nhân lên làm node gốc (tính sau cùng), đẩy phép cộng xuống tầng

con (buộc phải tính trước):

$$\begin{array}{ll}
 E \xrightarrow{lm} \underline{E} \times E & (\text{Chọn luật } E \rightarrow E \times E) \\
 \xrightarrow{lm} \underline{E} + E \times E & (\text{Thế biến trái nhất } \underline{E} \rightarrow E + E) \\
 \xrightarrow{lm} \text{id} + \underline{E} \times E & (\text{Thế biến trái nhất } \underline{E} \rightarrow \text{id}) \\
 \xrightarrow{lm} \text{id} + \text{id} \times \underline{E} & (\text{Thế biến trái nhất } \underline{E} \rightarrow \text{id}) \\
 \xrightarrow{lm} \text{id} + \text{id} \times \text{id} & (\text{Thế biến cuối cùng } \underline{E} \rightarrow \text{id})
 \end{array}$$

– **Trực quan hóa sự khác biệt cấu trúc cây:**

Cây cú pháp 1 (Diễn giải đúng toán học): Phép $+$ ở gần gốc hơn nên được tính toán sau cùng.

$$\begin{array}{c}
 E \text{ (Gốc - Tính sau cùng)} \\
 / \backslash \\
 E + E \\
 / \quad \backslash \\
 \text{id} \quad E * E \\
 \quad \quad / \quad \backslash \\
 \quad \quad \text{id} \quad \text{id}
 \end{array}$$

Biểu thức tương đương: $\text{id} + (\text{id} \times \text{id})$

Cây cú pháp 2 (Diễn giải sai toán học - Ép cộng trước): Phép $\times$ ở gần gốc hơn nên bị ép tính sau cùng, dẫn đến cụm bên dưới phải tính trước.

$$\begin{array}{c}
 E \text{ (Gốc - Tính sau cùng)} \\
 / \backslash \\
 E * E \\
 / \backslash \quad \backslash \\
 E + E \quad \text{id} \\
 / \quad \backslash \\
 \text{id} \quad \text{id}
 \end{array}$$

Biểu thức tương đương: $(\text{id} + \text{id}) \times \text{id}$

– **Kết luận:** Vì cùng một chuỗi w nhưng tồn tại hai dẫn xuất trái nhất dẫn đến hai cây phân tích cú pháp mang ngữ nghĩa khác nhau, văn phạm đã cho là văn phạm mơ hồ.

Dạng 3: Khử mơ hồ bằng phương pháp phân tầng văn phạm

• **Phương pháp giải:**

– **Nguyên lý tầng (Quy tắc đảo ngược):** Trong cây cú pháp, các phép toán được tính từ dưới (lá) lên trên (gốc). Do đó, toán tử có độ ưu tiên thấp ($+$, $-$) phải định nghĩa ở tầng cao nhất (gần ký hiệu bắt đầu S) để nó nằm gần gốc và được tính sau cùng. Toán tử độ ưu tiên cao ($\times$, $/$) phải định nghĩa dịch dần xuống các tầng thấp bên dưới để được gom cụm và tính trước.

– **Nguyên lý kết hợp:**

- * Toán tử kết hợp trái: Tạo luật đệ quy trái dạng $A \rightarrow A \text{ op } B \mid B$. Cấu trúc này ép cây phát triển lệch trái, giúp các toán tử bên trái nằm sâu hơn và được tính trước.
- * Toán tử kết hợp phải: Tạo luật đệ quy phải dạng $A \rightarrow B \text{ op } A \mid B$ (ép cây lệch phải).
- **Bài tập mẫu 2.3:** Khử mơ hồ cho văn phạm gồm hai toán tử $\{+, \times\}$ kết hợp trái tác động trên danh biểu id, biết $\times$ ưu tiên cao hơn $+$.
- **Lời giải chi tiết:** Áp dụng nguyên lý phân tầng để bề gãy tính phẳng của văn phạm ban đầu:
 - **Tầng 1 (E - Biểu thức chứa phép cộng):** Phép cộng $+$ ưu tiên thấp nhất $\Rightarrow$ Đặt ở tầng cao nhất để tính sau cùng. Kết hợp trái $\Rightarrow$ Đệ quy trái: $E \rightarrow E + T \mid T$
 - **Tầng 2 (T - Thuật ngữ chứa phép nhân):** Phép nhân $\times$ ưu tiên cao hơn $+$ $\Rightarrow$ Đặt ở tầng con dưới. Kết hợp trái $\Rightarrow$ Đệ quy trái: $T \rightarrow T \times F \mid F$
 - **Tầng 3 (F - Yếu tố cơ sở):** Chứa phần tử nguyên tố hoặc biểu thức ưu tiên tuyệt đối trong dấu ngoặc: $F \rightarrow \text{id} \mid (E)$
 - **Văn phạm hoàn chỉnh đã khử mơ hồ:**

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T \times F \mid F$$

$$F \rightarrow \text{id} \mid (E)$$

- **Bài tập mẫu 2.3b (Mở rộng biểu thức nhiều tầng):** Khử mơ hồ cho văn phạm phẳng chứa các toán tử toán học biểu thức bao gồm: phép cộng/trừ ($\{+, -\}$) kết hợp trái, phép nhân/chia ($\{\times, /\}$) kết hợp trái, toán tử một ngôi đổi dấu ($-$ định danh) và cặp dấu ngoặc cô lập (E). Biết thứ tự ưu tiên tăng dần là: $\{+, -\} \rightarrow \{\times, /\} \rightarrow$ Toán tử một ngôi $\rightarrow$ Dấu ngoặc.
- **Lời giải chi tiết:** Ta tiến hành bề gãy văn phạm phẳng thành 4 tầng cấu trúc từ độ ưu tiên thấp nhất đến cao nhất:
 - **Tầng 1 (E - Phép cộng và trừ):** Có độ ưu tiên thấp nhất, kết hợp trái $\Rightarrow$ Đệ quy trái trên cả hai toán tử:

$$E \rightarrow E + T \mid E - T \mid T$$

- **Tầng 2 (T - Phép nhân và chia):** Có độ ưu tiên trung bình, kết hợp trái $\Rightarrow$ Đệ quy trái trên cả hai toán tử:

$$T \rightarrow T \times F \mid T / F \mid F$$

- **Tầng 3 (F - Toán tử một ngôi):** Có độ ưu tiên cao hơn nhân/chia. Vì toán tử một ngôi đặt trước toán hạng (tiền tố) nên nó mang tính chất kết hợp phải $\Rightarrow$ Đệ quy phải:

$$F \rightarrow -F \mid P$$

- **Tầng 4 (P - Phần tử cơ sở):** Tầng đáy chứa định danh hoặc biểu thức bao bọc bởi dấu ngoặc để đạt độ ưu tiên tuyệt đối:

$$P \rightarrow \text{id} \mid (E)$$

Văn phạm hoàn chỉnh đã khử mơ hồ:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T \times F \mid T / F \mid F$$

$$F \rightarrow -F \mid P$$

$$P \rightarrow \text{id} \mid (E)$$

- **Bài tập mẫu 2.3d (Toán tử con trỏ và mảng):** Cho văn phạm phẳng, mơ hồ mô tả biểu thức chứa toán tử con trỏ (*) và toán tử truy xuất phần tử mảng ([]):

$$E \rightarrow *E \mid E[E] \mid \text{id}$$

Hãy phân tích độ ưu tiên và thực hiện khử mơ hồ cho văn phạm trên bằng phương pháp phân tầng.

- **Lời giải chi tiết:**

– **Phân tích độ ưu tiên và tính kết hợp:**

- * Theo quy ước ngôn ngữ lập trình, toán tử hậu tố truy xuất mảng $E[E]$ có độ ưu tiên cao hơn toán tử tiền tố một ngôi $*E$. Do đó, với chuỗi $*\text{id}[\text{id}]$, trình biên dịch phải hiểu là $*(\text{id}[\text{id}])$.
- * Toán tử mảng [] kết hợp trái $\Rightarrow$ Tạo luật đệ quy trái.
- * Toán tử con trỏ * kết hợp phải $\Rightarrow$ Tạo luật đệ quy phải.

– **Xây dựng hệ thống luật sinh phân tầng (Từ thấp đến cao):**

- * **Tầng 1 (E - Tầng thấp nhất về ưu tiên):** Chứa toán tử con trỏ * (ưu tiên thấp, tính sau cùng). Vì toán tử này đứng trước toán hạng nên nó mang tính chất kết hợp phải $\Rightarrow$ Đệ quy phải:

$$E \rightarrow *E \mid T$$

- * **Tầng 2 (T - Tầng ưu tiên cao hơn):** Chứa toán tử mảng []. Vì toán tử này kết hợp trái $\Rightarrow$ Sử dụng đệ quy trái ở biến hiệu hành:

$$T \rightarrow T[E] \mid F$$

Lưu ý: Bên trong cặp dấu ngoặc [] là một biểu thức độc lập hoàn chỉnh, do đó ta trả về ký hiệu bắt đầu E để có thể chứa bất kỳ biểu thức phức hợp nào làm chỉ số mảng (ví dụ: $\text{id}[*\text{id}]$).

- * **Tầng 3 (F - Tầng đáy cơ sở):** Chứa phần tử nguyên tố:

$$F \rightarrow \text{id}$$

– **Văn phạm hoàn chỉnh đã khử mơ hồ:**

$$E \rightarrow *E \mid T$$

$$T \rightarrow T[E] \mid F$$

$$F \rightarrow \text{id}$$

- **Kiểm chứng dẫn xuất trái nhất duy nhất cho chuỗi $w = *id_1[id_2]$:**

$E \xrightarrow{lm} *E$	(Bắt buộc chọn tầng con trở trước để tính sau cùng)
$\xrightarrow{lm} *T$	(Chuyển xuống tầng mảng)
$\xrightarrow{lm} *T[E]$	(Khai triển luật đệ quy trái của mảng)
$\xrightarrow{lm} *F[E]$	(Chuyển xuống tầng cơ sở)
$\xrightarrow{lm} *id_1[E]$	(Thế biến trái nhất $F \rightarrow id_1$)
$\xrightarrow{lm} *id_1[T]$	(Dẫn xuất biểu thức bên trong dấu ngoặc)
$\xrightarrow{lm} *id_1[F] \xrightarrow{lm} *id_1[id_2]$	

- **Bài tập mẫu 2.3e (Toán tử gọi hàm và toán tử một ngôi):** Cho văn phạm phẳng mô tả biểu thức chứa toán tử gọi hàm ($id()$) và toán tử logic NOT ($\neg$):

$$E \rightarrow \neg E \mid E(E) \mid id$$

Hãy phân tích độ ưu tiên và khử mơ hồ bằng phương pháp phân tầng.

- **Lời giải chi tiết:**

- **Phân tích:** Toán tử hậu tố gọi hàm $E(E)$ luôn có độ ưu tiên cao hơn toán tử tiền tố một ngôi $\neg E$. Ví dụ, biểu thức $\neg id(id)$ phải được hiểu là $\neg(id(id))$. Toán tử gọi hàm kết hợp trái, toán tử $\neg$ kết hợp phải.
- **Phân tầng luật sinh:**

$$\begin{aligned} E &\rightarrow \neg E \mid T && \text{(Tầng 1: Toán tử tiền tố } \neg \text{ ưu tiên thấp hơn, đệ quy phải)} \\ T &\rightarrow T(E) \mid F && \text{(Tầng 2: Toán tử gọi hàm ưu tiên cao hơn, đệ quy trái)} \\ F &\rightarrow id && \text{(Tầng 3: Định danh cơ sở)} \end{aligned}$$

Dạng 4: Vẽ Cây cú pháp (Parse Tree) và Cây cú pháp trừu tượng (AST)

- **Phương pháp giải:**

- Từ dẫn xuất trái nhất duy nhất của văn phạm phân tầng, vẽ chi tiết Parse Tree (giữ nguyên tất cả các biến trung gian nhằm thể hiện đúng cấu trúc ngữ pháp).
- Lược bỏ toàn bộ các biến trung gian (E, T, F) và dấu ngoặc không cần thiết để thu gọn thành AST (Node trong là toán tử, node lá là toán hạng) phục vụ trực tiếp cho việc tính toán giá trị.

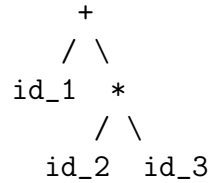
- **Bài tập mẫu 2.4:** Dựa vào văn phạm phân tầng ở Bài tập 2.3, hãy tìm dẫn xuất trái nhất và vẽ Parse Tree, AST cho chuỗi biểu thức: $w = id_1 + id_2 \times id_3$.

- **Lời giải chi tiết:**

- **Dẫn xuất trái nhất duy nhất (bắt buộc theo văn phạm phân tầng):** Do cấu trúc phân tầng, ta không thể chọn phép nhân trước, cấu trúc buộc phải triển khai phép cộng trước:

$$\begin{aligned} E &\xrightarrow{lm} E + T \xrightarrow{lm} T + T \xrightarrow{lm} F + T \xrightarrow{lm} id_1 + T \\ &\xrightarrow{lm} id_1 + T \times F \xrightarrow{lm} id_1 + F \times F \xrightarrow{lm} id_1 + id_2 \times F \\ &\xrightarrow{lm} id_1 + id_2 \times id_3 \end{aligned}$$

- Cấu trúc AST tinh gọn sau khi lược bỏ biến trung gian:



5 Kỹ thuật Phân tích LL(1)

Đây là trọng tâm của bài tập thực hành.

5.1 Điều kiện để một văn phạm là LL(1)

- Không đệ quy trái:** Nếu có văn phạm chứa đệ quy trái dạng $A \rightarrow A\alpha \mid \beta$, ta phải chuyển đổi thành:
 - $A \rightarrow \beta A'$
 - $A' \rightarrow \alpha A' \mid \epsilon$
- Đã đặt nhân tử chung trái:** Nếu có nhiều luật sinh chung tiền tố dạng $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$, ta phải đưa nhân tử chung ra ngoài thành $A \rightarrow \alpha A'$ và $A' \rightarrow \beta_1 \mid \beta_2$.

5.2 Các quy tắc cốt lõi tính toán tập hợp LL(1)

5.2.1 Quy tắc tìm tập FIRST

Tập $\text{FIRST}(\alpha)$ chứa các ký hiệu kết thúc (terminals) mở đầu cho các chuỗi được dẫn xuất từ α . Nếu α dẫn xuất ra chuỗi rỗng, thì ϵ thuộc $\text{FIRST}(\alpha)$.

- **Quy tắc 1 (Ký hiệu kết thúc):** Nếu X là ký hiệu kết thúc, thì $\text{FIRST}(X) = \{X\}$.
- **Quy tắc 2 (Ký hiệu rỗng):** Nếu có luật sinh $X \rightarrow \epsilon$, thì thêm ϵ vào $\text{FIRST}(X)$.
- **Quy tắc 3 (Biến chưa kết thúc):** Nếu X là biến chưa kết thúc và có luật sinh $X \rightarrow Y_1 Y_2 \dots Y_k$:
 - Thêm tất cả các ký hiệu kết thúc từ $\text{FIRST}(Y_1)$ (ngoại trừ ϵ) vào $\text{FIRST}(X)$.
 - Nếu $\text{FIRST}(Y_1)$ chứa ϵ , tiếp tục xét sang Y_2 và thêm $\text{FIRST}(Y_2) \setminus \{\epsilon\}$ vào $\text{FIRST}(X)$. Lặp lại cho đến khi gặp một Y_i không chứa ϵ .
 - Nếu tất cả các ký hiệu $Y_1, Y_2, \dots, Y_k$ đều chứa ϵ trong tập FIRST của chúng, lúc này ta mới thêm ϵ vào $\text{FIRST}(X)$.

5.2.2 Quy tắc tìm tập FOLLOW

Tập $\text{FOLLOW}(A)$ chứa các ký hiệu kết thúc có thể xuất hiện ngay sau biến A trong quá trình dẫn xuất chuỗi. *Lưu ý:* Tập FOLLOW không bao giờ chứa ký hiệu rỗng ϵ . Để tính FOLLOW cho các biến, ta quét qua tất cả các **vế phải** của các luật sinh:

- **Quy tắc 1 (Ký hiệu bắt đầu):** Nếu S là ký hiệu bắt đầu của văn phạm, luôn thêm ký hiệu kết thúc chuỗi $\$$ vào $\text{FOLLOW}(S)$.
- **Quy tắc 2 (Tìm biến đứng trước):** Nếu có luật sinh dạng $A \rightarrow \alpha B \beta$ (sau biến B là chuỗi β):
 - Lấy toàn bộ tập $\text{FIRST}(\beta)$ (ngoại trừ ϵ) thêm vào $\text{FOLLOW}(B)$.
- **Quy tắc 3 (Kế thừa từ biến cha):** Nếu có luật sinh dạng $A \rightarrow \alpha B$ (biến B đứng ở cuối cùng về phải) hoặc dạng $A \rightarrow \alpha B \beta$ mà tập $\text{FIRST}(\beta)$ có chứa ϵ :
 - Lấy toàn bộ tập $\text{FOLLOW}(A)$ (biến về trái) thêm vào $\text{FOLLOW}(B)$.

5.2.3 Quy tắc lập Bảng phân tích cú pháp LL(1)

Bảng phân tích cú pháp là một ma trận với các hàng là biến chưa kết thúc và các cột là ký hiệu kết thúc (cùng ký hiệu $\$$). Với mỗi luật sinh dạng $A \rightarrow \alpha$ trong văn phạm:

- Tìm tập $\text{FIRST}(\alpha)$. Với mỗi ký hiệu kết thúc $a \in \text{FIRST}(\alpha)$ (ngoại trừ ϵ), ta điền luật sinh $A \rightarrow \alpha$ vào ô tương ứng $T[A, a]$.
- Nếu tập $\text{FIRST}(\alpha)$ có chứa ϵ , ta tìm tập $\text{FOLLOW}(A)$. Với mỗi ký hiệu kết thúc $b \in \text{FOLLOW}(A)$ (bao gồm cả $\$$), ta điền luật sinh $A \rightarrow \epsilon$ vào ô tương ứng $T[A, b]$.

Dấu hiệu nhận biết: Nếu sau khi điền, có bất kỳ ô nào chứa từ 2 luật sinh trở lên, văn phạm đó xảy ra xung đột (đụng độ) và không phải là văn phạm LL(1).

5.3 Thuật toán mô phỏng Stack phân tích LL(1)

Bộ phân tích sử dụng một bộ đệm đầu vào chứa chuỗi cần phân tích (kết thúc bằng $\$$) và một Stack chứa các ký hiệu văn phạm (đáy stack là $\$$).

- **Khởi tạo:** $\text{Stack} = [\$, S]$ (với S là ký hiệu bắt đầu), Đầu vào = $[w \$]$.
- **Vòng lặp:** Gọi X là ký hiệu đỉnh Stack, a là token hiện tại ở đầu vào.
 - Nếu $X = a = \$ \Rightarrow$ Kết luận: Phân tích thành công (Chấp nhận chuỗi).
 - Nếu $X = a \neq \$ \Rightarrow$ Match thành công: Pop X ra khỏi Stack, tiến đầu vào lên một ký tự.
 - Nếu X là ký hiệu kết thúc (Terminal) nhưng $X \neq a \Rightarrow$ Báo lỗi cú pháp.
 - Nếu X là ký hiệu chưa kết thúc (Non-terminal): Tra bảng $T[X, a]$.
 - * Nếu $T[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$: Pop X ra khỏi Stack, sau đó Push các ký hiệu $Y_k, \dots, Y_2, Y_1$ vào Stack theo thứ tự ngược lại (để Y_1 nằm ở đỉnh Stack).
 - * Nếu $T[X, a] = X \rightarrow \epsilon$: Pop X ra khỏi Stack và không Push gì thêm.
 - * Nếu ô trống $\Rightarrow$ Báo lỗi cú pháp (Không có luật sinh phù hợp).

5.4 Hệ thống Bài tập Phân tích và So khớp Chuỗi (Theo mẫu bài giảng)

Bài tập 1: Phân tích văn phạm đặt nhân tử chung trái

Cho văn phạm G_1 gồm các luật sinh được đánh số:

- (1) $E \rightarrow TY$
- (2) $T \rightarrow \text{int}$
- (3) $T \rightarrow (E)$
- (4) $Y \rightarrow +E$
- (5) $Y \rightarrow \epsilon$

Yêu cầu: Kiểm tra tính chính quy, tìm tập FIRST, lập bảng LL(1) và so khớp chuỗi $w = \text{int} + \text{int}\$$.

Lời giải chi tiết:

1. Kiểm tra tính chính quy: Văn phạm G_1 **không phải** là văn phạm chính quy vì luật sinh (1) $E \rightarrow TY$ chứa hai biến liên tiếp ở vế phải, vi phạm dạng chuẩn loại 3 ($A \rightarrow wB$ hoặc $A \rightarrow w$).

2. Bảng biểu diễn tập FIRST:

E	T	Y
int	int	+
(	(	ϵ

Bảng biểu diễn tập FOLLOW:

E	T	Y
\$	+	\$
)	\$	)
	)	

Ghi chú giải thích chi tiết cách tìm tập FIRST và FOLLOW:

• **Tập FIRST:**

- FIRST(T): Do $T \rightarrow \text{int}$ và $T \rightarrow (E)$ nên nhận ký tự kết thúc đứng đầu là $\{\text{int}, (\}$.
- FIRST(E): Do $E \rightarrow TY$ nên lấy toàn bộ ký tự từ FIRST(T), suy ra FIRST(E) = $\{\text{int}, (\}$.
- FIRST(Y): Do $Y \rightarrow +E$ và $Y \rightarrow \epsilon$ nên nhận ký tự kết thúc là $\{+, \epsilon\}$.

• **Tập FOLLOW:**

- FOLLOW(E): Vì E là biến bắt đầu nên có \$. Do luật $T \rightarrow (E)$ nên E được theo sau bởi $)$, suy ra FOLLOW(E) = $\{ \$,) \}$.
- FOLLOW(Y): Xem luật $E \rightarrow TY$, vì Y đứng cuối vế phải nên lấy lại toàn bộ FOLLOW(E), suy ra FOLLOW(Y) = $\{ \$,) \}$.
- FOLLOW(T): Xem luật $E \rightarrow TY$, theo sau T là biến Y , ta lấy FIRST(Y) loại bỏ ϵ , tức là có $+$. Vì FIRST(Y) chứa ϵ (tức Y có thể biến mất), ta phải lấy thêm toàn bộ tập FOLLOW(E), suy ra FOLLOW(T) = $\{ +, \$,) \}$.

3. Bảng quy luật phân tích cú pháp LL(1) (Dạng số hiệu):

	int	+	(	)	\$
E	1		1		
T	2		3		
Y		4		5	5

4. Bảng mô phỏng Stack so khớp chuỗi nhập:

Cấu trúc Stack	Chuỗi nhập còn lại	Quy luật áp dụng / Hành động
$E \$$	int + int \$	Tra ô $T[E, \text{int}] = 1 \rightarrow$ Thay E bằng TY
$T Y \$$	int + int \$	Tra ô $T[T, \text{int}] = 2 \rightarrow$ Thay T bằng int
int $Y \$$	int + int \$	Match int: Gạch bỏ ở 2 bên.
$Y \$$	+ int \$	Tra ô $T[Y, +] = 4 \rightarrow$ Thay Y bằng $+E$
$+ E \$$	+ int \$	Match +: Gạch bỏ ở 2 bên.
$E \$$	int \$	Tra ô $T[E, \text{int}] = 1 \rightarrow$ Thay E bằng TY
$T Y \$$	int \$	Tra ô $T[T, \text{int}] = 2 \rightarrow$ Thay T bằng int
int $Y \$$	int \$	Match int: Gạch bỏ ở 2 bên.
$Y \$$	\$	Tra ô $T[Y, \$] = 5 \rightarrow$ Thay Y bằng ϵ
$\$$	\$	**Chấp nhận chuỗi (Thành công)**

Bài tập 1: Phân tích văn phạm chứa đệ quy kép

Cho văn phạm G_2 gồm các luật sinh được đánh số:

- (1) $S \rightarrow aB$
- (2) $S \rightarrow bA$
- (3) $S \rightarrow \epsilon$
- (4) $A \rightarrow aS$
- (5) $A \rightarrow bAA$
- (6) $B \rightarrow bS$
- (7) $B \rightarrow aBB$

Yêu cầu: Kiểm tra tính chính quy, tìm tập FIRST, lập bảng LL(1) và so khớp chuỗi nhập mẫu $w = abbs\$$.

Lời giải chi tiết:

1. Kiểm tra tính chính quy: Văn phạm G_2 **không phải** là văn phạm chính quy vì các luật sinh (5) $A \rightarrow bAA$ và (7) $B \rightarrow aBB$ xuất hiện hai biến liên tiếp ở vế phải.

2. Bảng biểu diễn tập FIRST:

S	A	B
a	a	b
b	b	a
ϵ		

Bảng biểu diễn tập FOLLOW:

S	A	B
\$	\$	\$
a	a	a
b	b	b

Ghi chú giải thích chi tiết cách tìm tập FIRST và FOLLOW:

• Tập FIRST:

- FIRST(S): Từ các luật $S \rightarrow aB$, $S \rightarrow bA$, $S \rightarrow \epsilon$, ta có ngay $\{a, b, \epsilon\}$.
- FIRST(A): Từ luật $A \rightarrow aS$ có a , luật $A \rightarrow bAA$ có b , suy ra $\{a, b\}$.
- FIRST(B): Từ luật $B \rightarrow bS$ có b , luật $B \rightarrow aBB$ có a , suy ra $\{a, b\}$.

• Tập FOLLOW: Do văn phạm đệ quy lồng chéo và tất cả các biến đều có thể đi kèm hoặc dẫn ra chuỗi chứa nhau nên cả 3 biến đều có chung tập kết thúc kết quả là $\{\$, a, b\}$.

- FOLLOW(S): Bắt đầu có $\$$. Xem luật $A \rightarrow aS$, theo sau S là đuôi của vế phải $\rightarrow$ lấy tiếp từ FOLLOW(A). Xem luật $B \rightarrow bS$, theo sau S cũng là đuôi $\rightarrow$ lấy tiếp từ FOLLOW(B).
- FOLLOW(A): Xem luật $S \rightarrow bA \rightarrow$ lấy từ FOLLOW(S). Xem luật $A \rightarrow bAA$, biến A thứ nhất được theo sau bởi biến A thứ hai $\rightarrow$ lấy FIRST(A) = $\{a, b\}$.
- FOLLOW(B): Xem luật $S \rightarrow aB \rightarrow$ lấy từ FOLLOW(S). Xem luật $B \rightarrow aBB$, biến B thứ nhất được theo sau bởi biến B thứ hai $\rightarrow$ lấy FIRST(B) = $\{a, b\}$.

3. Bảng quy luật phân tích cú pháp LL(1) (Dạng số hiệu):

	a	b	\$
S	1	2	3
A	4	5	
B	7	6	

4. Bảng mô phỏng Stack so khớp chuỗi nhập $w = abbs\$$:

Cấu trúc Stack	Chuỗi nhập còn lại	Quy luật áp dụng / Hành động
$S \$$	$a b b s \$$	Tra ô $T[S, a] = 1 \rightarrow$ Thay S bằng aB
$aB \$$	$b b s \$$	Match a: Gạch bỏ ở 2 bên.
$B \$$	$b s \$$	Tra ô $T[B, b] = 6 \rightarrow$ Thay B bằng bS
$bS \$$	$s \$$	Match b: Gạch bỏ ở 2 bên.
$S \$$	$s \$$	Tra ô $T[S, b] = 2 \rightarrow$ Thay S bằng bA
$bA \$$	$s \$$	Match b: Gạch bỏ ở 2 bên.
$A \$$	$s \$$	Đỉnh Stack là biến A , chuỗi nhập là $s \rightarrow$ Báo lỗi c

Bài tập 1: Phân tích văn phạm và khử đệ quy trái dạng chuỗi phức tạp

Cho văn phạm G_3 ban đầu gồm các luật sinh:

$$S \rightarrow uBDz$$

$$B \rightarrow Bv \mid w$$

$$D \rightarrow EF$$

$$E \rightarrow y \mid \epsilon$$

$$F \rightarrow x \mid \epsilon$$

Yêu cầu: Kiểm tra tính chính quy, tiến hành khử đệ quy trái tại B . Sau đó đánh số luật sinh văn phạm mới, tìm tập FIRST, lập bảng LL(1) và mô phỏng chuỗi $w = uvvyxz\$$.

Lời giải chi tiết:

1. Kiểm tra tính chính quy: Văn phạm G_3 **không phải** là văn phạm chính quy vì luật sinh $S \rightarrow uBDz$ kẹp hai biến ở giữa chuỗi kết thúc, và luật $D \rightarrow EF$ chứa hai biến đứng sát nhau.

2. Khử đệ quy trái trực tiếp tại biến B :

Luật sinh $B \rightarrow Bv \mid w$ có dạng đệ quy trái trực tiếp cấu trúc $\alpha = v$ và $\beta = w$. Tiến hành đưa về cấu trúc đuôi chuẩn (Tail-recursion) bằng cách thêm biến phụ B' :

$$(1) \quad S \rightarrow uBDz$$

$$(2) \quad B \rightarrow wB'$$

$$(3) \quad B' \rightarrow vB'$$

$$(4) \quad B' \rightarrow \epsilon$$

$$(5) \quad D \rightarrow EF$$

$$(6) \quad E \rightarrow y$$

$$(7) \quad E \rightarrow \epsilon$$

$$(8) \quad F \rightarrow x$$

$$(9) \quad F \rightarrow \epsilon$$

3. Bảng biểu diễn tập FIRST của văn phạm sau khi khử đệ quy:

S	B	B'	D	E / F
u	w	v	y	$E : y, \epsilon$
		ϵ	x	$F : x, \epsilon$
			ϵ	

Bảng biểu diễn tập FOLLOW của văn phạm sau khi khử đệ quy:

S	B	B'	D	E	F
$\$$	y	y	z	x	z
	x	x		z	
	z	z			

Ghi chú giải thích chi tiết cách tìm tập FIRST và FOLLOW:

• **Tập FIRST:**

- $\text{FIRST}(S) = \{u\}$, $\text{FIRST}(B) = \{w\}$, $\text{FIRST}(B') = \{v, \epsilon\}$, $\text{FIRST}(E) = \{y, \epsilon\}$, $\text{FIRST}(F) = \{x, \epsilon\}$ (theo kí tự đứng đầu luật sinh).
- $\text{FIRST}(D)$: Do $D \rightarrow EF$, ta lấy $\text{FIRST}(E)$. Vì $\text{FIRST}(E)$ có chứa ϵ , ta lấy tiếp $\text{FIRST}(F)$. Do cả E và F đều có ϵ nên D cũng nhận rỗng $\rightarrow \text{FIRST}(D) = \{y, x, \epsilon\}$.

• **Tập FOLLOW:**

- $\text{FOLLOW}(S) = \{\$ \}$ (biến bắt đầu).
- $\text{FOLLOW}(D)$: Xem luật $S \rightarrow uBDz$, ngay sau D là kí tự kết thúc $z \rightarrow \text{FOLLOW}(D) = \{z\}$.
- $\text{FOLLOW}(E)$: Xem luật $D \rightarrow EF$, sau E là $F \rightarrow$ lấy $\text{FIRST}(F)$ trừ đi ϵ , tức là có x . Vì F có chứa ϵ nên ta phải lấy thêm toàn bộ $\text{FOLLOW}(D) = \{z\} \rightarrow \text{FOLLOW}(E) = \{x, z\}$.
- $\text{FOLLOW}(F)$: Xem luật $D \rightarrow EF$, F đứng cuối vế phải $\rightarrow$ lấy lại toàn bộ $\text{FOLLOW}(D) = \{z\}$.
- $\text{FOLLOW}(B)$ và $\text{FOLLOW}(B')$: Xem luật $S \rightarrow uBDz$, theo sau B là biến $D \rightarrow$ lấy toàn bộ $\text{FIRST}(D)$ loại đi ϵ , thu được $\{y, x\}$. Nhưng vì $\text{FIRST}(D)$ có thể rỗng, ta bắt buộc phải xét tiếp ký tự liền sau D là kí tự z . Do đó $\text{FOLLOW}(B) = \{y, x, z\}$. Từ luật $B \rightarrow wB'$, biến B' đứng cuối vế phải nên $\text{FOLLOW}(B') = \text{FOLLOW}(B) = \{y, x, z\}$.

4. Bảng quy luật phân tích cú pháp LL(1) (Dạng số hiệu):

	u	v	w	x	y	z	\$
S	1						
B			2				
B'		3		4	4	4	
D				5	5	5	
E				7	6	7	
F				8		9	

5. Bảng mô phỏng Stack so khớp chuỗi nhập $w = uvvyxz\$$:

Cấu trúc Stack	Chuỗi nhập còn lại	Quy luật áp dụng / Hành động
$S \$$	$u w v y x z \$$	Tra ô $T[S, u] = 1 \rightarrow$ Thay S bằng $uBDz$
$\cancel{x} B D z \$$	$\cancel{x} w v y x z \$$	Match u: Gạch bỏ ở 2 bên.
$B D z \$$	$w v y x z \$$	Tra ô $T[B, w] = 2 \rightarrow$ Thay B bằng wB'
$\cancel{x} B' D z \$$	$\cancel{x} v y x z \$$	Match w: Gạch bỏ ở 2 bên.
$B' D z \$$	$v y x z \$$	Tra ô $T[B', v] = 3 \rightarrow$ Thay B' bằng vB'
$\cancel{x} B' D z \$$	$\cancel{x} y x z \$$	Match v: Gạch bỏ ở 2 bên.
$B' D z \$$	$y x z \$$	Tra ô $T[B', y] = 4 \rightarrow$ Thay B' bằng ϵ
$D z \$$	$y x z \$$	Tra ô $T[D, y] = 5 \rightarrow$ Thay D bằng EF
$E F z \$$	$y x z \$$	Tra ô $T[E, y] = 6 \rightarrow$ Thay E bằng y
$\cancel{y} F z \$$	$\cancel{y} x z \$$	Match y: Gạch bỏ ở 2 bên.
$F z \$$	$x z \$$	Tra ô $T[F, x] = 8 \rightarrow$ Thay F bằng x
$\cancel{x} z \$$	$\cancel{x} z \$$	Match x: Gạch bỏ ở 2 bên.
$\cancel{z} \$$	$\cancel{z} \$$	Match z: Gạch bỏ ở 2 bên.
$\$$	$\$$	**Chấp nhận chuỗi (Thành công)**